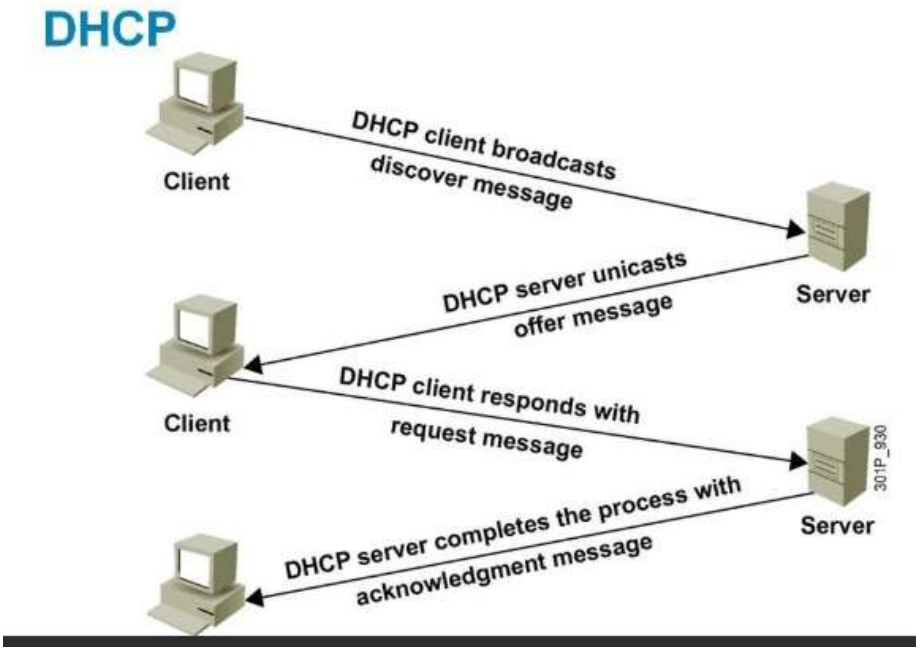
Here is a detailed, 12-mark explanation of the DHCP, FTP, and HTTP protocols, structured to highlight their core mechanisms, architectures, and functions.

1. Dynamic Host Configuration Protocol (DHCP) [4 Marks]

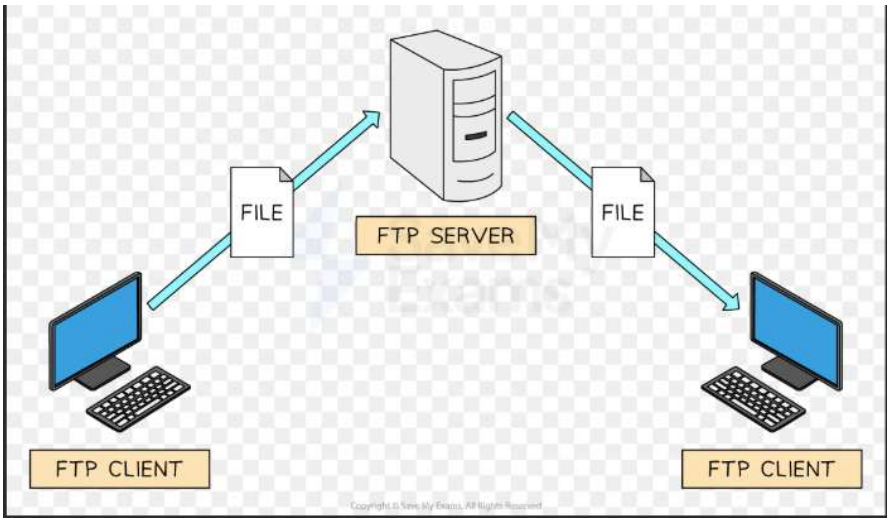
- **Purpose & Function [1 mark]:** DHCP is an application-layer protocol utilized to automatically assign IP addresses and essential network configuration parameters (such as Subnet Masks, Default Routers, and DNS addresses) to client devices on a network.
- **The DORA Process [1 mark]:** DHCP assigns IP addresses using a four-step process known as **DORA** (Discover, Offer, Request, Acknowledge).
- **Client Initialization (Discover & Request) [1 mark]:** When a client connects to the network, it lacks an IP address and broadcasts a **DHCP Discover** message to destination IP 255.255.255.255 to locate a DHCP server. Later in the process, the client broadcasts a **DHCP Request** message to formally accept a specific offered IP address and eliminate other potential server offers.
- **Server Response (Offer & Acknowledge) [1 mark]:** After receiving a Discover message, the server responds with a **DHCP Offer** containing an available IP address and lease duration. Once the client requests the offered IP, the server finalizes the assignment by sending a **DHCP Acknowledge (ACK)** message, confirming the IP and providing the subnet mask.



2. File Transfer Protocol (FTP) [4 Marks]

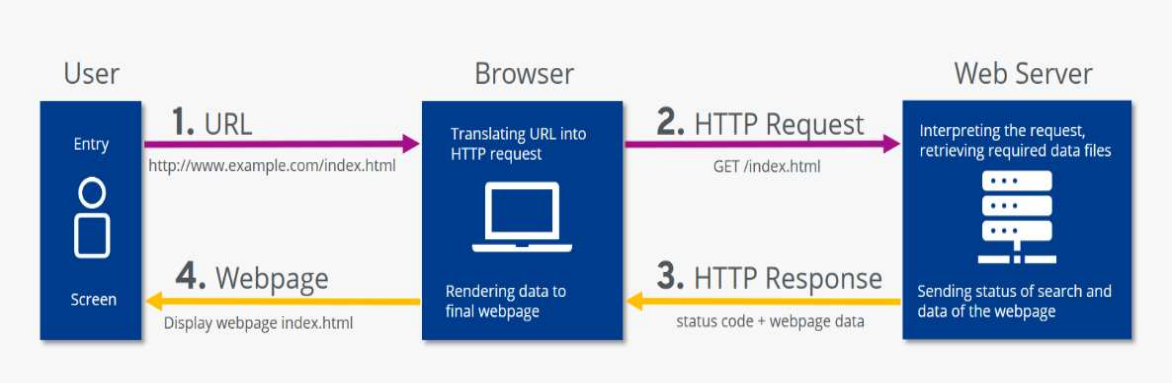
- **Purpose & Architecture [1 mark]:** FTP is a standard network protocol designed specifically to transfer computer files between a client and a server over TCP/IP networks.
- **Dual-Channel Operation [1 mark]:** Uniquely, FTP utilizes two distinct channels to operate. It establishes a **command channel** (typically on TCP port 21) exclusively for sending commands and receiving responses, and a separate **data channel** (typically on port 20) for the actual transfer of the files.

- **Active vs. Passive Modes [1 mark]:** FTP establishes data connections in two modes. In **Active mode**, the client tells the server which port it is listening on, and the server initiates the data connection. Because client-side firewalls often block this inbound connection, modern clients utilize **Passive mode (PASV)**, where the server provides a random port and the client initiates both the command and data connections.
- **Security Limitations [1 mark]:** Traditional FTP is inherently insecure because all transmissions, including login credentials (usernames and passwords) and file data, are sent in plain, unencrypted text. This vulnerability to packet sniffing has largely driven the adoption of secure alternatives like FTPS (which adds SSL/TLS) and SFTP (which uses Secure Shell).



3. Hypertext Transfer Protocol (HTTP) [4 Marks]

- **Purpose & Function [1 mark]:** HTTP is the foundational application-layer protocol of the World Wide Web, governing how web clients (browsers) request resources and how web servers respond and transfer data.
- **Request-Response Cycle [1 mark]:** Communication is entirely driven by a structured Request-Response cycle. The client establishes a TCP connection and sends an **HTTP Request** specifying a method (e.g., GET to retrieve data or POST to submit data), a URI, and headers. The server processes this and returns an **HTTP Response** containing a status code (e.g., 200 OK or 404 Not Found), response headers, and an optional data payload (like an HTML page or JSON object).
- **Statelessness [1 mark]:** HTTP is fundamentally a "stateless" protocol, meaning the server does not retain any memory of past client requests. To maintain user state across multiple transactions—such as keeping a user logged in or tracking a shopping cart—web applications must rely on external mechanisms like **cookies** and session IDs.
- **Protocol Evolution [1 mark]:** HTTP has evolved significantly to improve performance. **HTTP/1.1** introduced persistent connections but suffered from head-of-line blocking. **HTTP/2** improved throughput by introducing binary framing and multiplexing multiple streams over a single TCP connection. The latest standard, **HTTP/3**, replaces TCP entirely with QUIC (a UDP-based transport protocol) to eliminate transport-level blocking, reduce connection setup latency, and mandate TLS 1.3 encryption.



1. Definition and Significance (2 Marks)

Network Security refers to the set of policies, processes, and technologies designed to protect the integrity, confidentiality, and accessibility of computer networks and data.

- **Why it matters:** It protects sensitive information from theft, ensures regulatory compliance (like GDPR or HIPAA), and prevents operational downtime caused by cyberattacks.

2. The CIA Triad: Core Pillars (3 Marks)

To understand network security, one must look at the three foundational goals:

- **Confidentiality:** Ensuring data is accessible only to authorized users.
    - *Tools:* Encryption and Access Control Lists (ACLs).
- **Integrity:** Guaranteeing that data has not been altered or tampered with during transit.
    - *Tools:* Hashing and Digital Signatures.
- **Availability:** Ensuring that the network services and data are available to users whenever needed.
    - *Tools:* Redundancy and Load Balancing.

3. Key Components of Defense (4 Marks)

A robust security posture relies on multiple layers of defense:

- **Firewalls:** The first line of defense that monitors incoming/outgoing traffic based on security rules.
- **IDPS (Intrusion Detection and Prevention Systems):** Tools that monitor network traffic for suspicious activity and can actively block detected threats.
- **VPN (Virtual Private Network):** Creates a secure, encrypted "tunnel" for data transmission over public networks.
- **Network Access Control (NAC):** Restricts network access to only those devices that comply with security policies (e.g., having updated antivirus).

4. Common Threats (2 Marks)

Understanding the enemy is crucial. Major threats include:

- **Malware:** Viruses, worms, and Ransomware designed to damage systems.
- **Phishing:** Social engineering used to steal login credentials.
- **DDoS (Distributed Denial of Service):** Overwhelming a network to crash its services.
- **Man-in-the-Middle (MitM):** Attackers intercepting private communication between two parties.

5. Conclusion & Best Practices (1 Mark)

Effective security is not a "set it and forget it" task. It requires **Regular Software Updates**, **Multi-Factor Authentication (MFA)**, and **Employee Training** to mitigate the human element of risk.

Secure Sockets Layer (**SSL**) and its modern, more secure successor, Transport Layer Security (**TLS**), are cryptographic protocols designed to provide secure communication over a computer network. When you see a padlock icon in your browser, it means the connection is being protected by a TLS handshake.

The process works through a series of steps often referred to as the **TLS Handshake**, which combines both **Asymmetric** and **Symmetric** encryption.

1. The "Hello" Phase (Negotiation)

The process begins when a client (your browser) attempts to connect to a server.

- **Client Hello:** The client sends its TLS version and a list of supported "Cipher Suites" (algorithms it can use for encryption).
- **Server Hello:** The server responds by choosing the best TLS version and Cipher Suite from the list.

2. Authentication (The Certificate)

The server sends its **SSL/TLS Certificate** to the client.

- This certificate contains the server's **Public Key** and is signed by a trusted Certificate Authority (CA).
- The client verifies the certificate to ensure the server is who it claims to be, preventing Man-in-the-Middle (MitM) attacks.

3. Key Exchange (Asymmetric Encryption)

Once authenticated, the two parties need to agree on a "Master Secret" without anyone else seeing it.

- The client generates a random string of data (pre-master secret).
- It encrypts this data using the server's **Public Key** and sends it back.

- The server uses its **Private Key** to decrypt it. Now, both have the secret data, but no eavesdropper does.

4. Session Keys (Symmetric Encryption)

Asymmetric encryption (using public/private keys) is computationally expensive and slow for long-term data transfer.

- Both parties use the "Master Secret" to generate **Session Keys**.
- These are **Symmetric Keys**, meaning the same key is used to both encrypt and decrypt data. This is much faster for the actual exchange of information (like loading a webpage).

5. Secure Data Transmission

The handshake is "Finished." From this point forward, all data sent between the client and server is encrypted using the unique session keys. When the session ends (the browser tab is closed), those keys are discarded.

Cryptography is the science of securing information by transforming readable data (plaintext) into unreadable ciphertext using mathematical algorithms and keys. It ensures confidentiality, integrity, authentication, and non-repudiation, crucial for protecting digital communications, online transactions, and passwords. Modern cryptography includes symmetric key algorithms and asymmetric (public-key) techniques.

Key aspects of cryptography include:

- **Goals:**
    - **Confidentiality:** Ensuring only authorized parties can read the data.
    - **Integrity:**Ensuring data has not been altered in transit.
    - **Authentication:** Verifying the identity of the sender and receiver.
    - **Non-repudiation:** Preventing a sender from denying they sent a message
- Types of Cryptography:
- Symmetric-Key Cryptography: Uses the same secret key for both encryption and decryption.
- Asymmetric-Key Cryptography (Public-Key): Uses a public key for encryption and a different private key for decryption (e.g., RSA, Elliptic Curve Cryptography - ECC).

  Applications:

  Secure Web Browsing: HTTPS uses TLS/SSL protocols.

  Digital Signatures: Used to verify the authenticity of digital documents.

Cryptocurrency: Uses cryptographic hash functions and digital signatures to secure transactions.

The Symmetric Cipher Model

In classical cryptography, both the sender and receiver share a single, common secret key. The model consists of five essential components:

1. **Plaintext (X)**: The original, intelligible message.
2. **Encryption Algorithm (E)**: The mathematical process that transforms the plaintext.
3. **Secret Key (K)**: A value independent of the plaintext that dictates the algorithm's output.
4. **Ciphertext (Y)**: The scrambled, unreadable message produced. The relationship is defined as

   $$Y = E_K(X)$$

   .

5. **Decryption Algorithm (D)**: The reverse process used to recover the original message. The relationship is defined as

   $$X = D_K(Y)$$

   .

Core Techniques

- **Substitution**: Replacing elements of plaintext with other elements (e.g., Caesar Cipher).
- **Transposition**: Rearranging or shuffling the order of elements (e.g., Rail Fence Cipher).
- **Product Ciphers**: Modern systems like AES combine multiple rounds of both for maximum security.

---

2. Public Key (Asymmetric) Cryptography

Public Key Cryptography represents a fundamental shift from classical models by using a mathematically linked pair of keys: a **Public Key** and a **Private Key**.

Key Roles and Requirements

- **Public Key**: Distributed openly; anyone can use it to encrypt a message for the owner.
- **Private Key**: Kept strictly secret; it is the only key capable of decrypting messages encrypted with its corresponding public key.

- **Mathematical Hardness**: The system must rely on "hard problems" (like prime factorization in RSA), making it computationally infeasible to determine the private key even if the public key is known.

Primary Applications

1. **Confidentiality (Encryption)**: The sender uses the **receiver's public key** to encrypt. Only the receiver can read it, solving the "Key Distribution Problem".
2. **Digital Signatures (Authentication)**: The sender signs a hash of the message with their own **private key**. Anyone can use the sender's public key to verify it, providing authenticity and **non-repudiation**.

---

3. Comparison and Strategy

To secure top marks, it is essential to highlight the differences between these two branches:

Feature	Symmetric Encryption	Asymmetric (Public Key)
Number of Keys	1 (Shared Secret)	2 (Public & Private pair)
Key Exchange	High risk; requires secure channel	Low risk; public keys are public
Speed	Very Fast (ideal for bulk data)	Slower (computationally expensive)
Examples	AES, DES	RSA, Diffie-Hellman

TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection-oriented; uses a three-way handshake	Connectionless; no handshake
Guarantees reliable data delivery	Does not guarantee delivery

TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Uses acknowledgements (ACKs)	No acknowledgements
Supports retransmission of lost packets	No retransmission support
Ensures packets are delivered in order	Does not ensure ordering
Provides flow control and congestion control	No flow or congestion control
Slower due to higher overhead	Faster with minimal overhead
Variable header size (20–60 bytes)	Fixed header size (8 bytes)
Treats data as a continuous byte stream	Treats data as independent messages
Does not support broadcasting or multicasting	Supports broadcasting and multicasting
Used by HTTP, HTTPS, FTP, SMTP	Used by DNS, DHCP, VoIP, Streaming

Q1. Explain Routing Protocols

A **routing protocol** specifies the rules and mechanisms that routers use to communicate with each other and distribute information to select the most optimal paths between nodes on a computer network.

Without routing protocols, network paths would have to be manually configured in advance. By using routing protocols, routers can dynamically discover remote networks, maintain up-to-date routing tables, and automatically adapt to changing network conditions—such as routing data around failed links or congested pathways—which provides networks with high availability and fault tolerance.

The primary components of a dynamic routing protocol include:

- **Data structures:** Tables or databases kept in the router's RAM to store network information.
- **Routing protocol messages:** Messages used to discover neighboring routers, exchange routing data, and notify the network of topology changes.
- **Algorithms:** A finite list of mathematical steps used to process routing information and calculate the best path. Routing protocols use **metrics** (such as hop count, bandwidth, delay, or cost) to evaluate and differentiate between available paths to a destination.

Q2. Explain Types of Routing Protocols

Routing protocols are generally classified into three high-level categories:

1. Static Routing Protocol Routes are manually configured by a network administrator. Data packets follow fixed, predetermined paths that do not change unless manually updated. While this method is highly secure and requires minimal CPU processing, it is not scalable for large networks and cannot automatically adapt if a link fails.

2. Default Routing Protocol A simplified method that forwards all packets to a single, default exit point if the specific destination address is unknown. It is highly beneficial for "stub networks" (networks with only one possible exit route).

3. Dynamic Routing Protocol These protocols automatically and continuously adjust to changing networking conditions. They actively share route availability with other routers. Dynamic routing protocols are further subdivided by their **purpose** and their **operation**:

Classification by Purpose:

- **Interior Gateway Protocols (IGP):** Used for routing *within* a single Autonomous System (AS) or routing domain (a network under a single administrative control, like a company's internal network).
- **Exterior Gateway Protocols (EGP):** Used for routing *between* different Autonomous Systems (such as connecting a corporate network to an Internet Service Provider, or connecting ISPs together). BGP is the primary EGP used to run the Internet.

Classification by Operation (Algorithm):

- **Distance Vector Protocols:** Routers determine the best path by measuring the distance (metric) to a destination and the direction (next-hop vector). They act as "signposts" and only share their routing tables with their immediate neighbors, meaning they do not possess a complete map of the entire network. They are simpler to implement but suffer from slower convergence. Examples include **RIP** and **IGRP**.
- **Link-State Protocols:** Routers gather detailed link-state information from *all* other routers in the network, allowing every router to independently build a complete, identical map (topology) of the network. They use the Dijkstra Shortest Path First algorithm to calculate routes. They converge very quickly and are highly scalable. Examples include **OSPF** and **IS-IS**.

- **Hybrid / Advanced Distance Vector Protocols:** Cisco's **EIGRP** combines aspects of both types. It uses an advanced Diffusing Update Algorithm (DUAL) to maintain loop-free backup paths, allowing for nearly instantaneous convergence if a primary link fails.
- **Path-Vector Protocols:** Used by EGPs like **BGP**. Instead of simple metrics, they track the specific path that data packets take across multiple network domains, using attributes and routing policies to choose the best route and prevent loops.

Here is a comparison of Distance Vector Routing and Link State Routing, based on the provided sources:

Feature	Distance Vector Routing	Link State Routing
Network Knowledge	Operates on **local knowledge**, knowing only the distance and direction to a destination from its directly connected neighbors.	Operates on **global knowledge**, allowing every router to maintain a complete map of the entire network topology.
Algorithm Used	Uses the **Bellman-Ford algorithm** to calculate routes.	Uses the **Dijkstra algorithm** to calculate the shortest path.
Routing Updates	Sends **periodic copies of the entire routing table** exclusively to directly connected neighbors, regardless of whether a topology change occurred.	Sends **event-based, triggered updates** (Link State Packets) to all other routers in the network only when a topology change occurs.
Convergence Time	**Slow convergence**, meaning "negative news spreads slowly" and it takes longer to adapt to network changes.	**Fast convergence**, allowing it to quickly identify and adapt to network topology changes.
Resource Usage	Requires **less bandwidth, memory, and CPU power** because it does not flood the network and only processes local data.	Requires **higher bandwidth, memory, and CPU power** due to flooding updates and constantly calculating the shortest path across a complete database.
Routing Loops	Highly susceptible to **persistent routing loops** and the "count-to-infinity" problem.	Avoids the count-to-infinity problem and generally only experiences temporary loops.
Scalability	**Less scalable** due to strict hop limits (e.g., RIP allows 16 hops, IGRP allows 100).	**Highly scalable**, supporting infinite hops, making it the preferred choice for large networks.
Network Addressing	Historically acts as a **classful protocol**, meaning it does not support Variable Length Subnet Masks (VLSM) or Classless Inter-Domain Routing (CIDR).	Acts as a **classless protocol**, fully supporting VLSM and CIDR.
Metrics	Relies on metrics like **hop count and composite metrics**.	Relies on **cost** as its primary metric.
Common Examples	**RIP** and **IGRP**.	**OSPF** and **IS-IS**.